

JavaScript Array Method - shift()

The `shift()` method is one of the most commonly used array methods in JavaScript. It is used to **remove the first element** from an array.

Table of Contents

- What is shift()?
- Why Use shift()?
- Syntax
- Parameters
- Return Value
- How shift() Works
- Basic Examples
- Multiple Examples
- Real Project Examples
- Best Practices
- Common Mistakes
- Interview Questions
- Summary Table

What is shift()?

Definition

The `shift()` method removes the **first element** from an array.

It shifts all remaining elements one position to the left.

It modifies the **original array** and returns the **removed element**.

Why Use shift()?

We use `shift()` when we need to:

- Remove the first item

- Process a queue
- Remove the oldest notification
- Remove the first task
- Process API data one by one

Syntax

```
array.shift();
```

Parameters

`shift()` does **not** accept any parameters.

```
array.shift();
```

Return Value

`shift()` returns the **removed first element**.

Example

```
const fruits = ["Apple", "Mango", "Orange"];

let removed = fruits.shift();

console.log(removed);
```

Output

```
Apple
```

How shift() Works

Before

```
["Apple", "Mango", "Orange"]
```

↓

```
shift()
```

↓

Removed

```
Apple
```

↓

After

```
["Mango", "Orange"]
```

Notice that every remaining element moves one position to the left.

Example 1 (Basic)

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"
```

```
];  
  
fruits.shift();  
  
console.log(fruits);
```

Output

```
["Mango", "Orange"]
```

Explanation

- "Apple" is removed.
- "Mango" becomes the first element.
- "Orange" becomes the second element.
- The original array is modified.

Example 2 (Store Removed Element)

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
const removedFruit = fruits.shift();  
  
console.log(removedFruit);  
  
console.log(fruits);
```

Output

```
Apple  
  
["Mango","Orange"]
```

Example 3 (Numbers)

```
const numbers = [  
  10,  
  20,  
  30,  
  40  
];  
  
numbers.shift();  
  
console.log(numbers);
```

Output

```
[20,30,40]
```

Example 4 (Objects)

```
const users = [  
  {
```

```
        name: "John"

    },

    {

        name: "Sara"

    }

];

const removedUser = users.shift();

console.log(removedUser);

console.log(users);
```

Output

```
{ name: "John" }

[
  { name: "Sara" }
]
```

Example 5 (Empty Array)

```
const numbers = [];

const removed = numbers.shift();

console.log(removed);

console.log(numbers);
```

Output

```
undefined
```

```
[]
```

Example 6 (Multiple shift())

```
const colors = [  
  "Red",  
  "Blue",  
  "Green"  
];  
  
colors.shift();  
  
colors.shift();  
  
console.log(colors);
```

Output

```
["Green"]
```

Example 7 (Using Variables)

```
const tasks = [  
  "Study",  
  "Exercise",  
  "Sleep"
```

```
];  
  
let firstTask = tasks.shift();  
  
console.log(firstTask);
```

Output

```
Study
```

Example 8 (Single Element Array)

```
const items = [  
  "Laptop"  
];  
  
items.shift();  
  
console.log(items);
```

Output

```
[]
```

Real Project Example 1

Queue System


```
const queue = [
  "Customer 1",
  "Customer 2",
  "Customer 3"
];

const servedCustomer = queue.shift();

console.log(servedCustomer);

console.log(queue);
```

Output

```
Customer 1

["Customer 2","Customer 3"]
```

Real Project Example 2

Notification System

```
const notifications = [
  "Message",
  "Payment",
  "Order"
];

notifications.shift();
```

```
console.log(notifications);
```

Output

```
["Payment","Order"]
```

Real Project Example 3

Todo List

```
const todos = [  
  "Wake Up",  
  "Study",  
  "Exercise"  
];  
  
const completedTask = todos.shift();  
  
console.log(completedTask);  
  
console.log(todos);
```

Output

```
Wake Up  
  
["Study","Exercise"]
```

Real Project Example 4

Playlist

```
const playlist = [  
  "Song A",  
  "Song B",  
  "Song C"  
];  
  
playlist.shift();  
  
console.log(playlist);
```

Output

```
["Song B", "Song C"]
```

Real Project Example 5

Order Processing

```
const orders = [  
  "#1001",  
  "#1002",  
  "#1003"  
];
```

```
const processingOrder = orders.shift();

console.log(processingOrder);

console.log(orders);
```

Output

```
#1001

["#1002", "#1003"]
```

Best Practices

Use `shift()` only when removing the **first element**.

Store the removed element if it is needed later.

```
const firstItem = array.shift();
```

Check if the array is empty before calling `shift()` .

```
if(array.length > 0){

    array.shift();

}
```

Use meaningful variable names.

```
removedTask

servedCustomer
```

```
firstMessage
```

Common Mistakes

Mistake 1

Expecting `shift()` to return the updated array.

Wrong

```
const result = numbers.shift();  
  
console.log(result);
```

Output

```
10
```

It returns the **removed element**, not the updated array.

Correct

```
numbers.shift();  
  
console.log(numbers);
```

Mistake 2

Passing a parameter.

Wrong

```
numbers.shift(10);
```

`shift()` ignores all parameters.

Correct

```
numbers.shift();
```

Mistake 3

Using `shift()` on an empty array.

```
const arr = [];  
  
console.log(arr.shift());
```

Output

```
undefined
```

Mistake 4

Using the wrong method.

To remove the **last element**, use

```
pop()
```

instead of

```
shift()
```

Mistake 5

Forgetting that indexes change after `shift()` .

Before

```
["Apple", "Mango", "Orange"]
```

After

```
shift()
```

Result

```
["Mango", "Orange"]
```

Now,

```
array[0]
```

is

```
"Mango"
```

not `"Apple"` .

Interview Questions

What does `shift()` do?

It removes the **first element** from an array.

Does `shift()` modify the original array?

Yes.

What does `shift()` return?

The removed first element.

Does `shift()` take any parameters?

No.

What happens if the array is empty?

It returns

```
undefined
```

Which method removes the last element?

```
pop()
```

Which method adds an element at the beginning?


```
unshift()
```

What happens to the remaining elements after `shift()` ?

They move one position to the left.

Summary Table

Feature	Description
Method	<code>shift()</code>
Purpose	Remove the first element
Parameters	None
Returns	Removed first element
Changes Original Array	Yes
Removes From	Beginning of the array
Common Uses	Queue, Notifications, Todo List, Order Processing

Quick Comparison

Method	Action	Returns	Changes Original Array
<code>push()</code>	Add to end	New length	Yes
<code>pop()</code>	Remove from end	Removed element	Yes
<code>unshift()</code>	Add to beginning	New length	Yes
<code>shift()</code>	Remove from beginning	Removed element	Yes

Final Notes

The `shift()` method is commonly used in applications where the **oldest item needs to be processed first**. Typical use cases include:

- Queue systems (FIFO)
- Customer service queues
- Notification management
- Task processing
- Order processing
- Playlist management

Mastering `shift()` helps you understand queue-based data processing and prepares you for more advanced array operations.

JavaScript Array Method - unshift()

The `unshift()` method is one of the most commonly used array methods in JavaScript. It is used to **add one or more elements to the beginning** of an array.

Table of Contents

- What is unshift()?
- Why Use unshift()?
- Syntax
- Parameters
- Return Value
- How unshift() Works
- Basic Examples
- Multiple Examples
- Real Project Examples
- Best Practices
- Common Mistakes
- Interview Questions
- Summary Table

What is unshift()?

Definition

The `unshift()` method adds one or more elements to the **beginning** of an array.

It shifts all existing elements one position to the right.

It modifies the **original array** and returns the **new length** of the array.

Why Use unshift()?

We use `unshift()` when we need to:

- Add an item to the beginning
- Add the newest notification first
- Add a new task at the top
- Insert priority items
- Manage queue systems

Syntax

```
array.unshift(element1);  
  
array.unshift(element1, element2, element3);
```

Parameters

Parameter	Description
element1	First element to add
element2	Second element (Optional)
element3	Third element (Optional)

You can add one or more elements at once.

Return Value

`unshift()` returns the **new length** of the array.

Example

```
const fruits = ["Mango", "Orange"];

let length = fruits.unshift("Apple");

console.log(length);
```

Output

```
3
```

How unshift() Works

Before

```
["Mango", "Orange"]
```

↓

```
unshift("Apple")
```

↓

After

```
["Apple", "Mango", "Orange"]
```

All existing elements move one position to the right.

Example 1 (Basic)

```
const fruits = [  
  "Mango",  
  "Orange"  
];  
  
fruits.unshift("Apple");  
  
console.log(fruits);
```

Output

```
["Apple","Mango","Orange"]
```

Explanation

- **"Apple"** is added to the beginning.
- **"Mango"** moves to index **1** .
- **"Orange"** moves to index **2** .
- The original array is modified.

Example 2 (Numbers)

```
const numbers = [  
  20,  
  30,  
  40  
];  
  
numbers.unshift(10);
```

```
console.log(numbers);
```

Output

```
[10,20,30,40]
```

Example 3 (Multiple Elements)

```
const colors = [  
  "Green",  
  "Blue"  
];  
  
colors.unshift(  
  "Red",  
  "Black"  
);  
  
console.log(colors);
```

Output

```
["Red","Black","Green","Blue"]
```

Example 4 (Store Return Value)

```
const fruits = [  
  "Mango",  
  "Orange"  
];  
  
const newLength = fruits.unshift("Apple");  
  
console.log(newLength);  
  
console.log(fruits);
```

Output

```
3  
  
["Apple", "Mango", "Orange"]
```

Example 5 (Objects)

```
const users = [  
  {  
    name: "Sara"  
  }  
];  
  
users.unshift({  
  name: "John"  
});
```

```
console.log(users);
```

Output

```
[  
  { name: "John" },  
  { name: "Sara" }  
]
```

Example 6 (Empty Array)

```
const numbers = [];  
  
numbers.unshift(100);  
  
console.log(numbers);
```

Output

```
[100]
```

Example 7 (Mixed Data Types)

```
const data = [  
  
  true,  
  
  null  
  
];
```



```
data.unshift(  
  "Rokon",  
  23  
);  
  
console.log(data);
```

Output

```
["Rokon",23,true,null]
```

Real Project Example 1

Notifications

```
const notifications = [  
  "Payment Successful",  
  "Order Shipped"  
];  
  
notifications.unshift("New Message");  
  
console.log(notifications);
```

Output

```
[  
  "New Message",  
  "Payment Successful",
```

```
    "Order Shipped"  
  ]
```

Real Project Example 2

Todo List

```
const todos = [  
  "Exercise",  
  "Read Book"  
];  
  
todos.unshift("Study");  
  
console.log(todos);
```

Output

```
[  
  "Study",  
  "Exercise",  
  "Read Book"  
]
```

Real Project Example 3

Queue System

```
const vipQueue = [  
  "Customer 2",
```

```
    "Customer 3"

];

vipQueue.unshift("VIP Customer");

console.log(vipQueue);
```

Output

```
[
  "VIP Customer",
  "Customer 2",
  "Customer 3"
]
```

Real Project Example 4

Playlist

```
const playlist = [

  "Song B",

  "Song C"

];

playlist.unshift("Song A");

console.log(playlist);
```

Output

```
[  
  "Song A",  
  "Song B",  
  "Song C"  
]
```

Real Project Example 5

Recent Searches

```
const searches = [  
  "JavaScript",  
  "React"  
];  
  
searches.unshift("Node.js");  
  
console.log(searches);
```

Output

```
[  
  "Node.js",  
  "JavaScript",  
  "React"  
]
```

Best Practices

Use `unshift()` only when adding elements to the beginning.

Store the returned length if needed.

```
const length = array.unshift("New Item");
```

Use meaningful variable names.

```
notifications
```

```
tasks
```

```
messages
```

```
customers
```

Add multiple elements in one call if necessary.

```
numbers.unshift(1,2,3);
```

Common Mistakes

Mistake 1

Expecting `unshift()` to return the updated array.

Wrong

```
const result = numbers.unshift(10);  
  
console.log(result);
```

Output

```
4
```

It returns the **new array length**, not the array.

Correct

```
numbers.unshift(10);  
  
console.log(numbers);
```

Mistake 2

Using `unshift()` instead of `push()` .

Remember:

```
push()
```

adds to the **end**.

```
unshift()
```

adds to the **beginning**.

Mistake 3

Forgetting that indexes change.

Before

```
["Mango", "Orange"]
```

After

```
unshift("Apple")
```

Result

```
["Apple", "Mango", "Orange"]
```

Now,

```
array[0]
```

is

```
Apple
```

Mistake 4

Assuming the original array stays unchanged.

```
const arr = [2,3];  
  
arr.unshift(1);
```

Now,

```
arr
```

becomes

```
[1,2,3]
```

Interview Questions

What does `unshift()` do?

It adds one or more elements to the **beginning** of an array.

Does `unshift()` modify the original array?

Yes.

What does `unshift()` return?

The **new length** of the array.

Can `unshift()` add multiple elements?

Yes.

```
numbers.unshift(1,2,3);
```

Which method adds elements to the end?

```
push()
```

Which method removes the first element?


```
shift()
```

What happens to existing elements after `unshift()` ?

They move one position to the right.

Summary Table

Feature	Description
Method	<code>unshift()</code>
Purpose	Add element(s) to the beginning
Parameters	One or more elements
Returns	New array length
Changes Original Array	Yes
Adds To	Beginning of the array
Common Uses	Notifications, Queue, Todo List, Playlist

Quick Comparison

Method	Action	Returns	Changes Original Array
<code>push()</code>	Add to end	New length	Yes
<code>pop()</code>	Remove from end	Removed element	Yes
<code>unshift()</code>	Add to beginning	New length	Yes
<code>shift()</code>	Remove from beginning	Removed element	Yes

Final Notes

The `unshift()` method is useful when new items should appear **first** instead of last. It is commonly used in:

- Notification systems
- News feeds
- Chat applications
- Todo lists
- Priority queues
- Search history

Mastering `unshift()` helps you efficiently manage arrays where the newest or highest-priority items belong at the beginning.